FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

## Facultad de Defensa Nacional

## Carrera: Lic. en Ciberdefensa

# Materia

# Sistemas Operativos II

**Unidad 4 / Semana 10**

**Año:** 2025

## Contenidos

Interbloqueos (Deadlock): Ejemplo de Estrategia de Predicción: El Algoritmo del Banquero

## Índice

**1. Introducción** **3**
1.1 El Algoritmo del Banquero 3
**2. Funcionamiento del Algoritmo del Banquero** **4**
2.1 Conceptos Relacionados 4
2.1 Funcionamiento del algoritmo 5
2.2 Ejemplos 6
2.2.1 Estado Seguro 7
2.2.2 Estado Inseguro 9

# 1. Introducción

En el estudio de los sistemas operativos, el interbloqueo (o deadlock) representa una situación crítica en la que varios procesos quedan bloqueados indefinidamente al esperar recursos que nunca serán liberados.
Se ha visto que para abordar este problema, existen cuatro estrategias fundamentales: prevención, que evita que se cumplan las condiciones necesarias para que ocurra el interbloqueo; evitación o predicción[1], que analiza dinámicamente cada solicitud de recursos para garantizar que el sistema permanezca en un estado seguro (como lo hace el algoritmo del banquero); detección, que permite que el interbloqueo ocurra pero lo identifica mediante algoritmos específicos; y recuperación, que actúa una vez detectado el interbloqueo, liberando recursos o reiniciando procesos, además del ya mencionado *algoritmo del avestruz,* que prioriza ignorar el problema y espera una intervención manual para su resolución.
De todas estas estrategias, en esta oportunidad nos vamos a centrar en una muy conocida y estudiada que es el ***algoritmo del banquero***. Si bien no es un algoritmo que se use actualmente en los sistemas operativos modernos, es muy útil desde el punto de vista didáctico para comprender el funcionamiento de este tipo de algoritmos.

## 1.1 El Algoritmo del Banquero

El algoritmo del banquero fue propuesto por Edsger W. Dijkstra en 1965, como parte de sus investigaciones pioneras sobre concurrencia y asignación segura de recursos en sistemas operativos. Lo presentó en una de sus célebres notas técnicas, la *EWD108*, escrita originalmente en holandés, donde abordaba cómo evitar interbloqueos mediante decisiones preventivas en la concesión de recursos. Este algoritmo fue concebido en el contexto del desarrollo del sistema operativo *THE*, uno de los primeros en implementar una arquitectura por capas, diseñado en la década de 1960 en la Technische Hogeschool Eindhoven (actual Universidad Técnica de Eindhoven), de donde toma su nombre.

Dijkstra eligió el nombre "**algoritmo del banquero**" por la analogía con una situación típica del ámbito financiero: en ella, los procesos se asemejan a clientes que solicitan préstamos, y los recursos disponibles representan el dinero que el banco puede prestar.

En este escenario, el banco cuenta con una *reserva limitada de fondos* y una lista de clientes, cada uno con una línea de crédito previamente establecida. En cualquier momento, un cliente puede solicitar una parte de su crédito, sin garantía de que realizará un reembolso hasta haber recibido el monto total autorizado.

Ante esta incertidumbre, el banquero puede optar por rechazar una solicitud si considera que, al concederla, podría quedarse sin fondos suficientes para atender futuras solicitudes que sí permitirían a los clientes completar sus operaciones y devolver el dinero.

Esta lógica preventiva es la base del algoritmo, que busca garantizar que el sistema permanezca siempre en un **estado seguro**, evitando así posibles bloqueos.

[1] Hay una controversia en algunos autores respecto de la palabra "predicción". De hecho, se podrían considerar las estrategias estudiadas en esta sección como ejemplos de prevención del interbloqueo debido a que impiden efectivamente la aparición del interbloqueo pero no trabajan sobre sus causas, sino que sobre la posibilidad de que se produzcan en cada asignación de recursos. Estas estrategias también son conocidas como de "evasión" de interbloqueos

# 2. Funcionamiento del Algoritmo del Banquero

## 2.1 Conceptos Relacionados

Antes de ver los detalles del funcionamiento del algoritmo del banquero, vamos a definir algunos conceptos importantes para entender el funcionamiento de este algoritmo.

**Estado del sistema**

El algoritmo utiliza el concepto de ***estado del sistema*** y lo clasifica en dos grupos:

- **Estado seguro**

  Es la situación en la que existe al menos **una secuencia de ejecución** que permite que todos los procesos finalicen sin quedar bloqueados.

- **Estado inseguro**

  Decimos que el sistema está en estado inseguro cuando no se puede garantizar que los procesos puedan completar su ejecución. No implica que haya interbloqueo pero puede conducir al él.

**Secuencia segura**

Orden de ejecución de los procesos en el que cada uno puede obtener sus recursos, ejecutarse y liberar recursos para el siguiente sin generar interbloqueos.

**Proceso**

Entidad que solicita y utiliza recursos del sistema para ejecutar tareas. En el algoritmo, cada proceso debe declarar su demanda máxima de recursos antes de comenzar.

**Recurso**

Unidad de servicio que puede ser asignada a procesos (ej.: CPU, memoria, archivos, impresoras). Se clasifican por tipo y cantidad disponible.

**Recursos Disponibles (Available)**

Normalmente se almacena en un vector que indica cuántas unidades de cada recurso están actualmente libres en el sistema en un momento dado.

**Demanda máxima (Max)**

Es una Matriz que representa el *número máximo de unidades de cada recurso que cada proceso podría solicitar durante su ejecución.*

**Asignación actual (Allocation)**

Es la Matriz que indica *cuántas unidades de cada recurso han sido asignadas actualmente a cada proceso.*

**Necesidad restante (Need)**

Es la Matriz calculada como:

**Necesidad restante (Need)** = **Demanda máxima (Max)** - **Asignación actual (Allocation)**

Indica cuántos recursos adicionales necesita cada proceso para completar su ejecución.

**Solicitud de recursos**

Petición que realiza un proceso para obtener recursos adicionales. Antes de concederla, el sistema simula la asignación y verifica si el estado resultante es seguro.
Si la simulación da como resultado que el sistema sigue en *estado seguro se acepta*, de lo contrario se *rechaza temporalmente evitando quedar en estado inseguro*.

**Simulación de asignación**

Proceso interno del algoritmo en el que se simula la concesión de recursos para verificar si el sistema permanece en estado seguro luego de aceptar la solicitud.

## 2.1 Funcionamiento del algoritmo

Paso a paso del algoritmo

**Declaración inicial**

Cada proceso debe informar su demanda máxima de recursos (**Max**). El sistema también conoce cuántos recursos están disponibles (**Available**) y cuántos ya han sido asignados (**Allocation**).

**Cálculo de necesidades restantes**

Se calcula la matriz **Need** como:

**Need** = **Max** -**Allocation** - Esto indica cuántos recursos adicionales necesita cada proceso para finalizar.

**Solicitud de recursos**

Cuando un proceso solicita recursos, el sistema verifica si la solicitud *es menor o igual* a su *Need* y si los recursos están disponibles (*Available*). Si no se cumple alguna de estas condiciones, la solicitud se rechaza.

**Simulación de asignación**

Si la solicitud es válida, el sistema simula conceder los recursos y actualiza temporalmente las matrices Available, Allocation y Need.

**Verificación de estado seguro**

Se intenta encontrar una secuencia segura de ejecución:

Se busca un proceso cuyo *Need* sea menor o igual a los recursos disponibles (*Available*).

Si se encuentra, se simula su ejecución: libera sus recursos y se suman a Available.

Se repite el proceso hasta que todos los procesos puedan finalizar.

**Decisión final**

Si se encuentra **una secuencia segura**, es decir que todos los procesos pueden finalizar en la simulación, entonces la **solicitud se concede**.

Si no, **se rechaza temporalmente para evitar entrar en un estado inseguro que podría conducir a un interbloqueo.**

## 2.2 Ejemplos

Veamos a continuación ejemplos en que el sistema obtiene un *estado inseguro* y la solicitud de asignación se deniega y el caso de un sistema seguro en donde la mencionada solicitud se acepta y se continúa con el procesamiento.

A continuación se presentan dos ejemplos prácticos utilizando tres recursos (*A, B y C*) y cuatro procesos (*P0, P1, P2 y P3*). El primero ilustra una situación en la que el sistema alcanza un *estado seguro* tras simular la asignación de recursos, mientras que el segundo muestra un escenario en el que no es posible garantizar una secuencia segura de ejecución, lo que conduce a un *estado inseguro*.

Total de recursos disponibles inicialmente:  Total Recursos= **[A=10, B=5, C=7]**

**Max**

| **Proceso** | **A** | **B** | **C** |
|---|---|---|---|
| P0 | 7 | 5 | 3 |
| P1 | 3 | 2 | 2 |
| P2 | 9 | 0 | 2 |
| P3 | 2 | 2 | 2 |

## 2.2.1 Estado Seguro

**Allocation**

| Proceso | A | B | C |
|---|---|---|---|
| P0 | 0 | 1 | 0 |
| P1 | 2 | 0 | 0 |
| P2 | 3 | 0 | 2 |
| P3 | 2 | 1 | 1 |

**Available**

Cálculos de los recursos disponibles:

Suma de Allocation:

A: 0+2+3+2 = 7

B: 1+0+0+1 = 2

C: 0+0+2+1 = 3

Total - Suma:

Available = [10-7, 5-2, 7-3] = **[3, 3, 4]**

**Need**

**Need = Max - Allocation**

| Proceso | A | B | C |
|---|---|---|---|
| P0 | 7 | 4 | 3 |
| P1 | 1 | 2 | 2 |
| P2 | 6 | 0 | 0 |
| P3 | 0 | 1 | 1 |

**Algoritmo de Verificación**

Disponible [3,3,4]
Finish[0,0,0,0] (Finalización de procesos)

**P0:** No puede finalizar por falta de recursos

**P1:** Sí puede finalizar, Need ≤ Available → [1, 2, 2] ≤ [3, 3, 4]

- Simula terminación: libera su Allocation: [2, 0, 0]
- Available: [5, 3, 4]
- Finish[1] = [0,1,0,0]

**P3:** Sí puede finalizar, Need ≤ Available → [0, 1, 1] ≤ [5, 3, 4]

- Libera [2, 1, 1]
- Available: [7, 4, 5]
- Finish[3] = [0,1,0,1]

**P0:** Sí puede finalizar, Need [7, 4, 3] ≤ [7, 4, 5]

- Libera [0, 1, 0]
- Available: [7, 5, 5]
- Finish[0] = [1,1,0,1]

**P2:** Need [6, 0, 0] ≤ [7, 5, 5]

- Libera [3, 0, 2]
- Available: [10, 5, 7]
- Finish[2] = [1,1,1,1]

Comprobamos con esta simulación **que todos los procesos pueden finalizar con esta asignación** lo que determina que el estado del **sistema es SEGURO**.

## 2.2.2 Estado Inseguro

**Allocation**

| Proceso | A | B | C |
|---|---|---|---|
| P0 | 3 | 1 | 0 |
| P1 | 2 | 1 | 1 |
| P2 | 3 | 0 | 2 |
| P3 | 2 | 3 | 1 |

**Available**

Cálculos de los recursos disponibles:

Suma de Allocation:

A: 3+2+3+2 = 10

B: 1+1+0+3 = 5

C: 0+1+2+1 = 4

Total - Suma:

Available = [10-10, 5-5, 7-4] = **[0, 0, 3]**

**Need**

**Need = Max - Allocation**

| Proceso | A | B | C |
|---|---|---|---|
| P0 | 4 | 4 | 3 |
| P1 | 1 | 1 | 1 |
| P2 | 6 | 0 | 0 |
| P3 | 0 | 1 | 1 |

**Algoritmo de Verificación**

Disponible **[0,0,3]**
Finish[0,0,0,0] (Finalización de procesos)

**P0:** No puede finalizar por falta de recursos [4,4,3] > [0,0,3]

**P1:** No puede finalizar por falta de recursos [1,1,1,] > [0,0,3]

**P2:** No puede finalizar por falta de recursos [6,0,0] > [0,0,3]

**P3:** No puede finalizar por falta de recursos [0,1,1] > [0,0,3]

**No hay proceso que pueda finalizar**, el estado queda en **estado INSEGURO.**



***Actividad S10-A1***

Realice la actividad identificada como S10-*A1* (Semana 10, Actividad 1) en la sección de actividades del campus.